

Oblig 4

Teori

Oppgave 1.1 - Ord og begreper

Skriv en oversikt over hva følgende ord/begreper/teknologier betyr/er:

- *Interface*

Et *interface* er tilnærmet en *abstrakt* klasse, men enda strengere. Et *interface* er en helt abstrakt klasse som ikke kan inneholde noen form for kode kropp i sin metoder. Det kan heller ikke opprettes noen objekter direkte av denne klassen. De kan altså ikke inneholde en konstruktør eller instans variabler. Videre kan vi ikke arve et *interface*, men vi kan implementere en klasse med et *interface* og videre arve interfacet sine egenskaper fra denne klassen.

- *Exceptions*

I Java er alle *exceptions* representert gjennom klasser. Alle *Exception* klasser tilhører i et hierarki hvor klassen som heter *Throwable* er root, eller top av klasse hierarkiet. Klassen *Throwable* har to direkte subklasser, som er *Error* og *Exception*. *Error* klassen håndterer systemfeil som oppstår i JVM systemet og ikke i koden vi har laget. Med dette så menes det at dette er feil som er utenfor en utvikler sin kontroll og at dette ikke kan rettes opp i det skrevne programmet. Feil som oppstår grunnet feil i kodestrukturen håndteres gjennom *Exception* klassen. Eksempler på slike feil kan være tilfeller hvor systemet prøver å dele på null, array begrensninger og avvik i programmets filer. En viktig subklasse til *Exceptions* er *RunTimeExceptions* som brukes til å representere forskjellige typer av vanlige feil som kan oppstå under kjøring av programmet. Vi kan videre bruke dette til å overskrive den originale *RunTimeError* feilmeldingen og selv definere hva som skal skrives ut ved en gitt feil i kjøringen.

I Java *Exception* håndtering benytter vi oss av fem nøkkelord som hører tett sammen i en slik håndtering:

- *Try og catch*: Her definerer vi hvilken kodeblokk vi ønsker å gjøre en *Exception* sjekk på. Om programmet går som forventet vil kjøringen gå videre, men om det oppstår en feil som har blitt definert i *catch* blokken vil det *catch* unntaket som passer med feilmeldingen bli kjørt. Om feilmeldingen ikke passer med noe av det som er blitt tatt høyde for i *catch* blokken vil en systemmelding fra JVM systemet bli printet.
- *Throw og Throws*: Disse to nøkkelordene fungerer my likt som *try* og *catch*, men i stedet for å innhenge *Exception* håndteringen rundt kodeblokken, så bruker vi denne løsningen for å implementere en eller flere *Exceptions* håndteringer inne i kode blokken. *Throw* kan implementeres i metoder og blokker gjennom if-statements, mens *throws* bakes inn i en metode under deklarasjon av metode som en forlengelse av deklarasjonen.
- *Finally*: Her kan vi legge til en kodeblokk som blir kjørt uavhengig av om koden i *try* blokken faller inn under en *Exception* eller ikke.

- *Hva er de tre forskjellige typene exceptions i Java. Gi gjerne eksempler på hvilke tilfeller hver av dem kan forekomme. Forklar også hvordan vi programmerere må forholde oss til exceptions når vi skriver kode.*
- *JVM internal error:* Dette er feil i systemet som er utenfor programmerers kontroll og må håndteres ved å sjekke at systemfiler til JVM er korrekte(re-install, update, osv.).
- *Standard exceptions:* Her snakker vi om typiske logiske feil som å dele på null, NULL i en string verdi, eller lignende.
- *Throw:* Dette er manuelt genererte *exceptions* som har blitt lagt inn i koden under deklarasjon eller som if-statements.

Når vi programmerer er det viktig å ha med *Exception* håndtering i programmene av flere grunner. De to mest essensielle er forståelse og sikkerhet. Når en bruker tar i bruk programvaren er det ikke sikkert at de har samme kompetansenivå som de som utviklet systemet har. Dermed er det viktig at bruker får tilbakemeldinger som er forståelige og gir mening, slik at de får mulighet til å innrette seg på en måte slik at de klarer å bruke systemet riktig basert på innholdet i feilmeldingen. Når det kommer til sikkerhet, så kan en standard *RunTimeError* tilbakemelding gi ut informasjon om systemet som utvikler ikke ønsker skal være synlig for en bruker. Denne informasjonen kan potensielt brukes til å hente ut sensitiv informasjon, eller alterere kodestrukturen som fører til at programmet endrer adferd i forhold til hva som var den originale hensikten med programmet.